CS4433 Software Engineering: Final Project Report

Moderated Learning Management System (LMS)
Kabelo Helen Khobotle, student number 202322667

Development model: Waterfall with V-Model verification and validation
Git repository: https://github.com/KhobotleHelen/SoftwareEngineering.git
All images, graphs and tables are separate files in the repository folders report/images, report/graphs and report/tables

1 Abstract and contents

This report covers the life cycle of a Moderated Learning Management System for a secondary school: planning, requirements, design, implementation, testing, quality assurance, deployment and maintenance. The system serves four roles (administrator, lecturer, Head of Department and student) and enforces the rule that students see no mark until the Head of Department has approved and released it. It is a C++17 console application with SQLite storage, works offline, hashes passwords with Argon2id and keeps an audit log of every mark change. It is verified by 122 automated tests with 98.4% line coverage of the business-logic layer, clean static analysis and a passing CI pipeline. Contents: 2 Introduction, 3 Project management, 4 Requirements, 5 Design, 6 Implementation, 7 Verification and validation, 8 Configuration management and quality assurance, 9 Deployment, 10 Maintenance, 11 Professional, legal and ethical issues, 12 Conclusion, 13 References, 14 Appendices.

2 Introduction

A secondary school needs an LMS for courses, assignments and assessment. Users are students, lecturers, Heads of Department (HoDs) and an administrator, each limited to their role. Each submission follows a life cycle (submitted, late, marked, moderated, released) and every mark change must be recorded. Stakeholders gave conflicting statements: the Principal wants simplicity and HoD-checked marks, the lecturer wants quick marks and extensions, the HoD wants a late-work penalty and sight of all marks, the student representative wants resubmission and written feedback, and the IT officer needs an offline, secure and fast system for 800 students on old computers. Payment, video conferencing, automatic grading, plagiarism detection and web or mobile interfaces are out of scope. The project follows Waterfall over seven weeks, with V-Model test levels, using C++17, CMake, SQLite, libsodium, Git and GitHub Actions, GoogleTest, cppcheck, gcovr and lizard.

3 Project management plan

The feasibility study found the project technically feasible (C++17 with SQLite, offline), operationally feasible (four clear roles, simple menus), economically feasible (no extra services; costs must be confirmed by the school), legally feasible with conditions (children's personal data under the Data Protection Act 2012, the Children's Protection and Welfare Act 2011 and the Persons with Disability Equity Act 2021) and feasible within seven weeks although tight. The work breakdown has nine packages: planning, requirements, design, implementation, verification and validation, configuration management and QA, deployment, maintenance, and completion. The Gantt chart shows the dependencies, the three phase gates and the two releases; the critical path runs from requirements through design, implementation, testing, deployment and maintenance. Six risks were registered with likelihood, impact, exposure, mitigation and owner (old computers, unreliable network,

requirement conflicts, user difficulty, disputed business rules, too little testing time). After implementation the register stands as: network and requirement conflicts closed; computers, rules and testing reduced; user difficulty still open until the usability test is done. See Table 21, the risk register, and the feasibility table.

4 Requirements engineering

Seven conflicts or gaps in the stakeholder statements were resolved: marks are hidden until the HoD releases them; extensions are recorded by the lecturer and change the deadline used for lateness; late work is accepted for up to three days with 10% of the maximum mark deducted per day, and after that is accepted but scores zero (a part-day counts as a full day); students may resubmit before the deadline and the latest version is marked; written feedback is mandatory with every mark; the system is offline with salted password hashes; and menus are simple and role-specific (conflict table). The SRS (ISO/IEC/IEEE 29148) has 15 functional requirements (FR-01 to FR-14 in v1.0, FR-15 class-wide extension in v1.1) and 8 measurable non-functional requirements, for example login within 2 seconds with 800 users stored, passwords only as salted hashes, and no crash on bad input, each prioritised with MoSCoW (functional and non-functional tables). The use case diagram covers the four actors with include, extend and generalisation; three use cases (Submit assignment, Create assignment, Review and release marks) are fully specified in the Phase 2 document.

5 Software design

A layered architecture was chosen (presentation, business logic, data access, SQLite) because it separates concerns, lets access checks run before protected operations, keeps all SQL in one place and supports offline use; a single mixed program was rejected because changes would spread and database access could not follow consistent security rules (component diagram). The class diagram has an abstract `User` with four role subclasses, domain classes, services and repository interfaces. The Strategy pattern isolates the late-penalty rule, the Repository pattern keeps SQL out of business classes, and a Factory (added during implementation) builds the right user subclass; the design follows the SOLID principles. Behaviour is modelled by the submission state machine, a swimlane activity diagram and two sequence diagrams (submit, moderate and release). The database has ten tables in Third Normal Form (ERD). Implementation exposed gaps that were corrected: a `head_id` column on `Department` to tie an HoD to a department, two triggers and seven indexes, and the `UserFactory` (design changes).

6 Implementation

The code (about 2,900 lines in `src`) follows the layers: `ui` (console menus, no SQL or rules), `services` (authentication, access control, users, academic data, assignments, submissions, assessment), `strategy`, `domain`, `data` (repository interfaces and SQLite implementations) and `security`. Access control combines role permissions with scoped checks (a lecturer must teach the course, a student must be enrolled, an HoD must head the department). `AssessmentService` applies the late penalty, stores the final mark and writes an audit row (who, when, old mark, new mark, reason) in the same transaction as the mark change; a failed audit write rolls the mark back. Students never receive a mark or feedback that is not released. SQLite is used only through the data-access layer with parameterised queries, foreign keys on, and triggers that stop a mark exceeding the maximum. Passwords are hashed with Argon2id through libsodium with a random 16-byte salt

per password and a constant-time comparison; it was chosen because it is memory-hard, widely reviewed and recommended by OWASP, unlike fast hashes such as SHA-256, and it works offline on all platforms. Hashing takes about 55 ms, so login stays far below 2 seconds. A single exception hierarchy and one guard in the UI turn every error into a short message instead of a crash. The build uses CMake with a documented coding standard and the build guide. Known limitations: file paths are stored rather than copies, there is no login lock-out or time-out, the database is not encrypted, and approved marks cannot be re-opened.

7 Verification and validation

The V-Model matches unit tests to the class design, integration tests (real in-memory SQLite) to the architecture and database, scripted console system tests to the use cases and SRS, and acceptance tests to the stakeholder rules and non-functional requirements, each with entry and exit criteria (test strategy). Equivalence partitioning and boundary value analysis cover the deadline and late-penalty rules (deadline minus 1 second, exactly the deadline, plus 1 second, plus 3 days, plus 3 days and 1 second), marks (below 0, 0, maximum, above maximum) and password length. The suite has 122 tests in 22 suites, all passing; line coverage is 98.4% for the business-logic layer (745 of 757 lines, target 70%) and 92.7% for all code except `main`. System tests cover first-run setup, the whole submit, mark, moderate and release flow, late penalties, individual and class extensions, role access, invalid input and a deliberately broken database (system tests). The defect log has four build-time defects (a CMake syntax error, a missing include, one wrong test expectation, and 12 style findings), all fixed (defect log). Still to do before real use: the NFR-07 usability test with one user per role (sheet) and repeating the 800-student performance test on the school's own computer.

8 Configuration management and quality assurance

Git is used with a documented strategy: `main` always builds, work happens on feature branches merged by pull request, commits are small, and releases are tagged v1.0 and v1.1 (workflow). A GitHub Actions pipeline builds with warnings as errors, runs all tests, fails if business-logic coverage drops below 70% and runs cppcheck (CI run). cppcheck reported 12 style findings in v1.0, all fixed, and none in v1.1. Metrics: 2,946 lines, 237 functions, average cyclomatic complexity 2.6, maximum 14 (limit 15). The self code review against a checklist led to nine changes, such as making `AccessControl::require` static, removing a dead function and replacing raw loops with standard algorithms (evidence files).

9 Deployment

The system runs on one school computer (or one computer reached over the school network): the `lms` executable and the `lms.db` SQLite file on the computer, linked to SQLite and libsodium, and an optional shared folder holding the files named in submissions (deployment diagram). A technician installs it by installing the build tools, building with CMake, running the 122 tests, choosing a backed-up folder for the database and creating the administrator account on first start (installation guide). Step-by-step manuals for the four roles, with real program output, are in the user manuals, and the release notes describe v1.0 and v1.1.

10 Maintenance (CR1)

CR1 asked for class-wide deadline extensions that are shown on the student's assignment list. It is perfective maintenance: nothing was faulty, users wanted a new capability and a better display. The impact analysis predicted changes to the requirements (new FR-15, FR-06 extended), one use case, `AssignmentService`, the student list view, the console menu and tests, and no change to tables or to the penalty and submission logic; the actual change matched (impact analysis, full document). The new method `grantClassExtension` extends every enrolled student in one transaction (all or nobody), never shortens a later individual extension, and records the reason as a class extension; the student list now shows `EXTENDED to <date>` using the same `DeadlineResolver` as the penalty calculation. Regression testing ran all 112 v1.0 tests (one menu script line updated), which passed, plus 10 new tests for the change, giving 122 passing tests, unchanged 98.4% coverage and no new static-analysis findings (new tests).

11 Professional, legal and ethical issues

The system holds children's personal data, so it collects only a username and role, limits every role to the records it needs, keeps data on the school's computer with no internet transfer, and should be re-checked against the Data Protection Act 2012 and the Children's Protection and Welfare Act 2011 with the school (retention periods and correction requests are the school's decisions). Assessment records are protected by salted Argon2id hashes, parameterised SQL, the audit log, hidden unreleased marks and locked approved marks; the remaining risks are the unencrypted database file and the lack of login lock-out. The console interface uses plain text, numbered menus, no colour and no time limits, which suits screen readers and keyboard use, but it has no own large-text settings and has not been tested with learners with disabilities (Persons with Disability Equity Act 2021). The late-penalty rule is applied identically to everyone and is auditable, but counting a part-day as a full day, taking 10% of the maximum rather than of the mark earned and awarding zero after three days can be harsh; extensions (individual or, since v1.1, for a whole class when a problem affects everyone), accepting late work, and showing the lecturer the penalty reduce the harm. The work follows the ACM Code of Ethics by stating limitations and reporting results honestly.

12 Conclusion

The Moderated LMS was taken from planning to a tested C++17 application, released as v1.0 and changed to v1.1 through a controlled change request. The layered design, the Strategy, Repository and Factory patterns, the audit log and the Argon2id password storage meet the stakeholder needs, and 122 automated tests, 98.4% coverage, clean static analysis and a green CI pipeline support the claims. Implementation exposed gaps that design reviews missed, and the design then paid off in maintenance: CR1 touched three classes, needed no database change and left all earlier tests passing. Remaining work is the usability test, performance confirmation on the school's computers, login lock-out and time-out, keeping every version of a resubmission, and re-opening approved marks with a recorded reason.

13 References

ACM (2018) *ACM Code of Ethics and Professional Conduct*. New York: ACM. Biryukov, A., Dinu, D. and Khovratovich, D. (2016) 'Argon2: new generation of memory-hard functions for password hashing', *IEEE European Symposium on Security and Privacy*, pp. 292-302. Gamma, E. et al. (1994)

Design Patterns. Reading, MA: Addison-Wesley. ISO/IEC/IEEE (2018) *ISO/IEC/IEEE 29148:2018 Requirements engineering*. Geneva: ISO. Kingdom of Lesotho (2011) *Children's Protection and Welfare Act, 2011*; (2012) *Data Protection Act, 2012*; (2021) *Persons with Disability Equity Act, 2021*. Maseru: Government of Lesotho. Martin, R.C. (2017) *Clean Architecture*. Boston: Prentice Hall. OWASP (n.d.) *Password Storage Cheat Sheet*, https://cheatsheetseries.owasp.org (Accessed: 1 November 2026). Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson. libsodium (n.d.) *Password hashing*, https://doc.libsodium.org (Accessed: 1 November 2026). SQLite (n.d.) *Documentation*, https://www.sqlite.org/docs.html (Accessed: 1 November 2026).

14 Appendices (in the repository)

Full 41-page report with all figures and tables | Traceability matrix | Test results | Coverage | Regression results | Design changes and limitations | User manuals | All images, graphs and tables